



Proven Patterns for Building Enterprise Vue Apps

Ben Hong

<https://www.bencodezen.io>



Introductions

Who am I?



UNIVERSAL'S ISLANDS OF ADVENTURE™

November 12, 2017



ISLAND
ADVENTURE



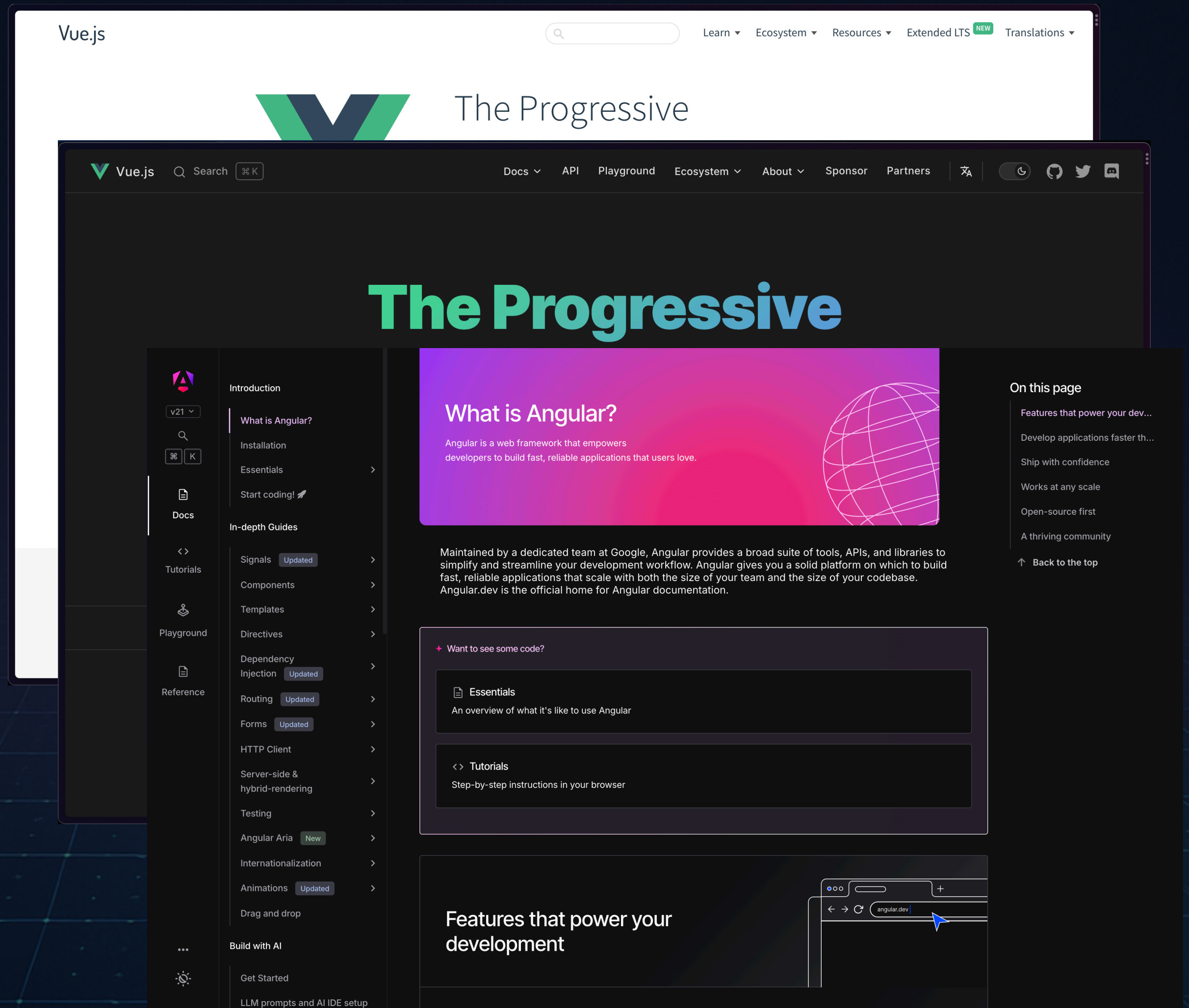
Ben Hong

Staff DX Engineer

Vue Core Team

Angular Team

@bencodezen



Who are you?

Who are you?

Fill out the following survey:

<https://form.typeform.com/to/nYUFj948>

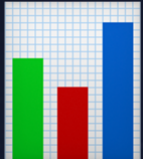


 **Hi there, neighbor!**

 **Hi there, neighbor!**

1. What's their name?
2. What's their role?
3. What are their goal(s) for this workshop?

 **Hi there, neighbor!**

 Survey results! 



Setup

 **Why are we here?**



What will we cover?



What will we cover?

- Languages
- Components
- Routing
- Reusability & Composition
- State Management
- Testing
- Tooling
- Fostering Change
- AI



Schedule

Time Slot	Event
9:00AM - 9:30AM	Introductions
9:30AM - 10:30AM	AI, Languages
10:30AM - 10:40AM	☕ Short Break 🍵
10:40AM - 12:00PM	Components
12:00PM - 1:00PM	🥪🥗 Lunch 🍟🥤
1:00PM - 2:30PM	Routing + Reusability & Composition
2:30PM - 2:40PM	🍪 Short Break 🍫
2:40PM - 4:00PM	R&C, State Management & Testing
4:00M - 4:10PM	🧁 Short Break 🍰
4:10PM - 4:45PM	Fostering Change, AI
4:45PM - 5:00PM	Closing Thoughts + Q&A

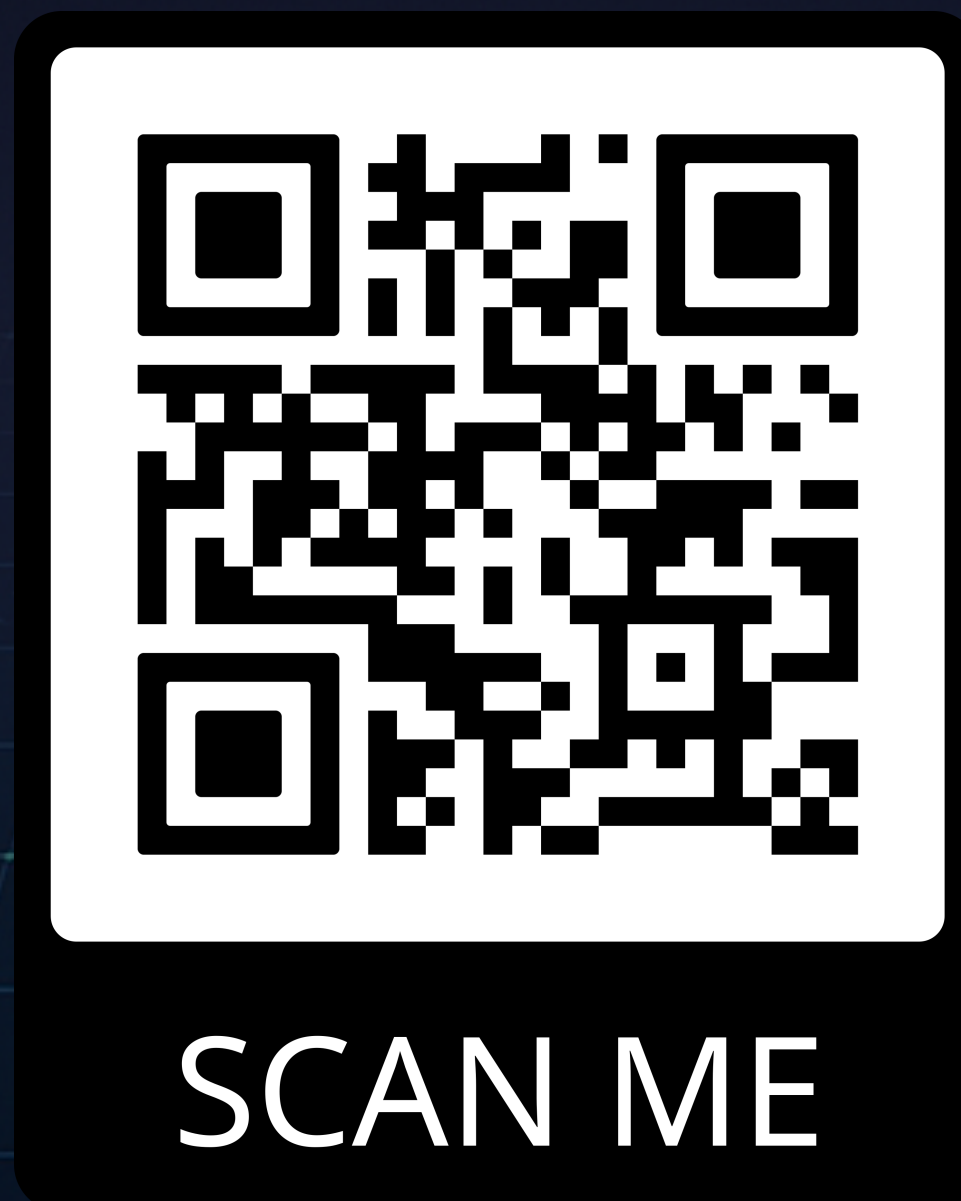


What do you need?

Resources

Proven Patterns Workshop Repo

<https://github.com/bencodezen/proven-patterns-workshop>





Resources

Your Projects

I encourage you to pull from your experience to make the most of this workshop.



Workshop Format

Workshop Format

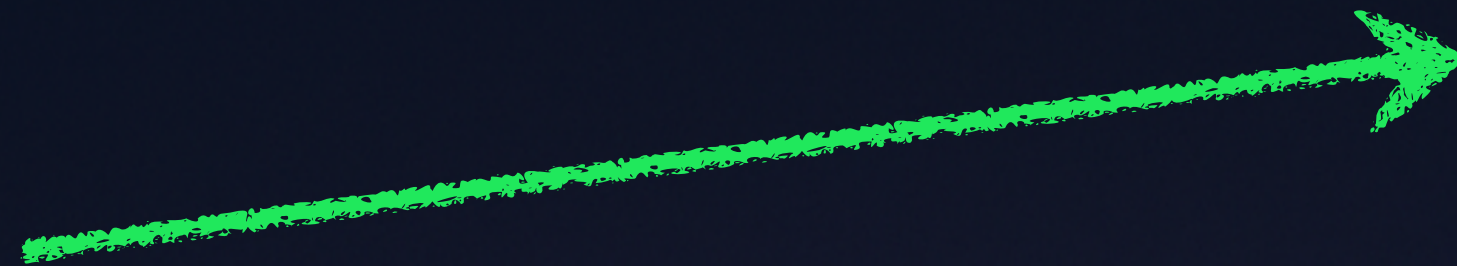
1. Learn

2. Question

3. Apply

Workshop Format

1. Learn



Explanations

Examples

Stories

2. Question



Clarifications

What-ifs

3. Apply



Code

Experiment

Pairing

Participation Tips

Participation Tips

This is a **safe place** to learn from one another

<https://vuejs.org/about/coc.html>

Participation Tips

Questions are **welcome!**

Discussions are how we can make the most of our time together.

Participation Tips

All slides and examples will be **public**.

No need to hurriedly copy down everything!

Participation Tips

Please **no recording.**

Out of respect for you and your participants' privacy

Participation Tips

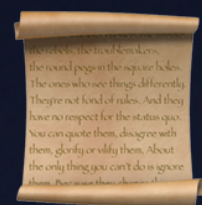
If you need a **break**, please take one!

Or if you need to leave for any other reason, feel free to.

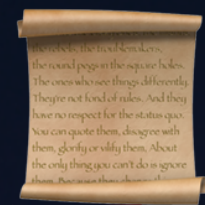
Participation Tips

Raise your hand for **questions** at any time.

Discussions are how we can make the most of our time together.



Key Principles

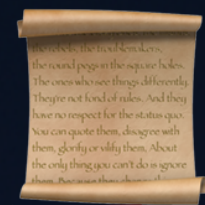


Key Principles

All code is **compromise**.

I encourage you to question and/or disagree.

Your opinion and experience matter.



Key Principles

There are no “best practices,”
only **chosen conventions**.

Every team has different trade-offs they’re willing to accept.

Choose what works best for you and your team.



AI Primer

A scene from the movie Toy Story featuring Woody and Buzz Lightyear. Woody, on the left, is a cowboy doll with a yellow and red plaid shirt, a white shirt with black cow print, and a brown belt with a large gold buckle. He has a concerned expression. Buzz Lightyear, on the right, is a space ranger doll in a green and white suit with purple accents. He is holding a green blaster and has a confident, slightly smug expression. The background is a simple indoor setting with a door and some furniture.

AI HERE!

AI THERE!

AI EVERYWHERE!!!

**IT'S LIKE NORMAL TOOTHPASTE BUT
WE ADDED AI**





What is our relationship with
AI exactly?

It's not your mentor...

It's not your intern...

It's not really your co-worker either...

It's not your mentor...

It's not your intern...

It's not really your co-worker either...

It's more like your “second brain.”

It's more like your “**second brain.**”

Conventions and **patterns** are
more important than ever.



Languages



HelloVueConfUS.vue

```
<template>
  <h1>Hello VueConfUS 2026!</h1>
</template>
```

```
<script>
export default {
  data: () => ({
    location: 'Atlanta, GA'
  })
}
</script>
```

```
<style>
h1 {
  background: white;
  color: blue;
}
</style>
```



HelloVueConfUS.vue

```
<template>
  <h1>Hello VueConfUS 2026!</h1>
</template>

<script setup lang="ts">
import { ref } from 'vue'

const location = ref('Atlanta, GA')
</script>

<style>
h1 {
  background: white;
  color: blue;
}
</style>
```

LANGUAGES

HTML

TECHNIQUE

Template vs Render Function

TECHNIQUE

Template



MyFirstComponent.vue

```
<template>
  <h1 class="title">Hello VueConfUS!</h1>
</template>
```

```
<script>
export default {
  // ...
}
</script>
```

```
<style>
/* My Custom Styles */
</style>
```

TECHNIQUE

Render Function



MyFirstComponent.vue

```
<template>
  <h1 class="title">Hello VueConfUS!</h1>
</template>

<script>
export default {
  // ...
}
</script>

<style>
/* My Custom Styles */
</style>
```

TECHNIQUE

Render Function



MyFirstComponent.vue

```
<template>
  <h1 class="title">Hello VueConfUS!</h1>
</template>

<script>
import { h } from 'vue'

export default {
  // ...
}
</script>

<style>
/* My Custom Styles */
</style>
```

TECHNIQUE

Render Function



MyFirstComponent.vue

```
<template>
  <h1 class="title">Hello VueConfUS!</h1>
</template>

<script>
import { h } from 'vue'

export default {
  render() {
    return h()
  }
}
</script>
```

TECHNIQUE

Render Function



MyFirstComponent.vue

```
<script>
import { h } from 'vue'

export default {
  render() {
    return h('h1', { class: 'title' }, 'Hello VueConfUS!')
  }
}
</script>

<style>
/* My Custom Styles */
</style>
```

TECHNIQUE

Render Function



MyFirstComponent.vue

```
<script setup lang="ts">
import { h } from 'vue'

const renderTemplate = h('h1', { class: 'title' }, 'hello')
</script>

<style>
/* My Custom Styles */
</style>
```

TECHNIQUE

Render Function



MyFirstComponent.vue

```
<script setup lang="ts">
import { h } from 'vue'

const renderTemplate = h('h1', { class: 'title' }, 'hello')
</script>

<template>
  <component :is="renderTemplate" />
</template>

<style>
/* My Custom Styles */
</style>
```

TECHNIQUE

Template vs Render Function

- Templates are the most declarative way to write HTML and is recommended as the default
- Render functions are a valid alternative to templates that are great for programmatic generation of markup (e.g., libraries, etc.)

CONVENTION

All HTML should exist in .vue files as a
<template> or render function.

BEST PRACTICE

All HTML should exist in .vue files as a `<template>` or render function.

- A benefit of doing this is that Vue has an opportunity to parse it before the browser does

BEST PRACTICE

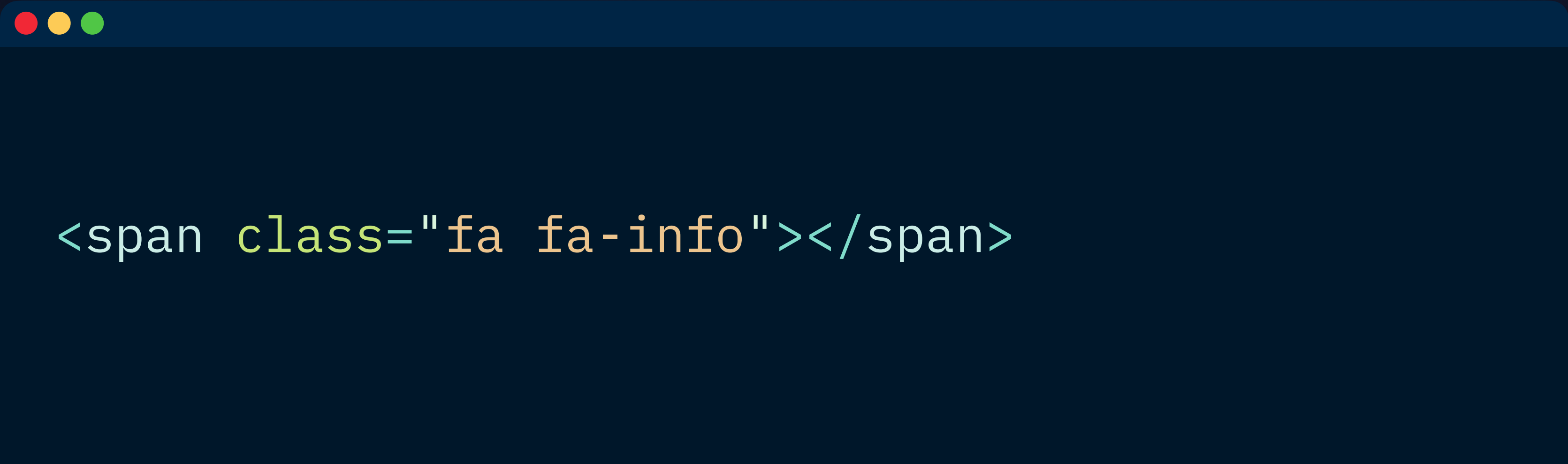
All HTML should exist in .vue files as a `<template>` or render function.

- A benefit of doing this is that Vue has an opportunity to parse it before the browser does
- This allows for developer experience improvements such as:
 - Self-closing tags

BEST PRACTICE

All HTML should exist in .vue files as a `<template>` or render function.

- Self-closing tags

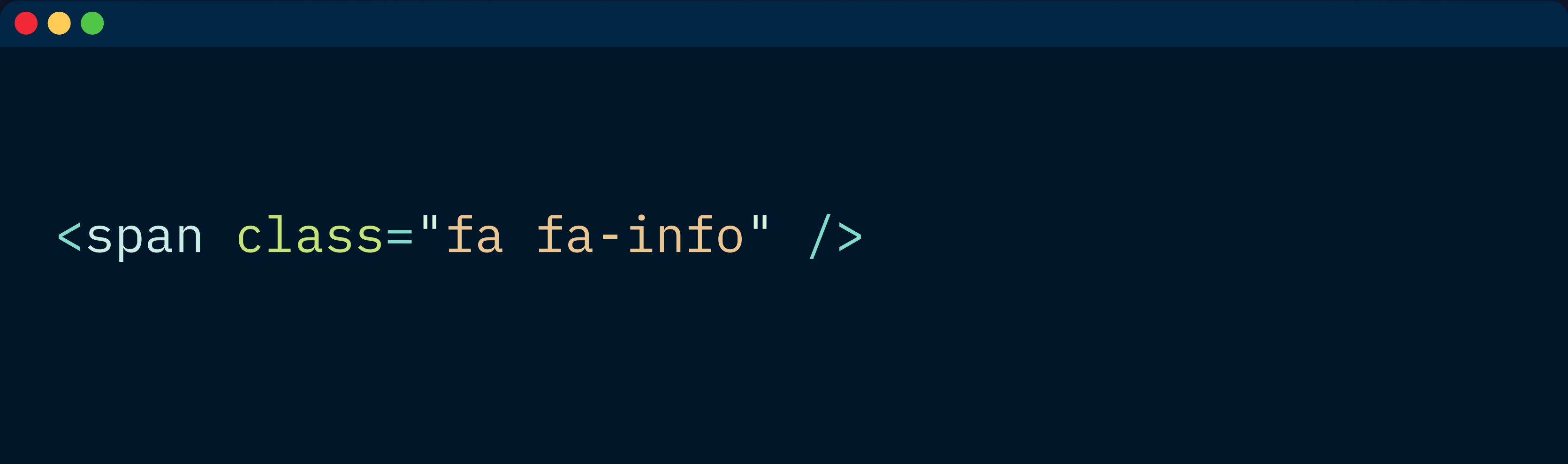
A dark-themed code editor window with a title bar containing three colored window control buttons (red, yellow, green). The editor area displays a single line of HTML code: `<span class="fa fa-info"></span>`. The code is color-coded: `<` is light blue, `span` is light green, `class="` is yellow, `fa fa-info` is orange, `>` is light blue, `</` is light blue, and `span>` is light green.

```
<span class="fa fa-info"></span>
```

BEST PRACTICE

All HTML should exist in .vue files as a `<template>` or render function.

- Self-closing tags



```
<span class="fa fa-info" />
```

BEST PRACTICE

All HTML should exist in .vue files as a `<template>` or render function.

- Self-closing tags

```
<BaseIcon  
  name="info"  
  size="large"  
  text="Information Icon"  
></BaseIcon>
```

BEST PRACTICE

All HTML should exist in .vue files as a `<template>` or render function.

- Self-closing tags

```
<BaseIcon  
  name="info"  
  size="large"  
  text="Information Icon"  
>
```

BEST PRACTICE

All HTML should exist in .vue files as a `<template>` or render function.

- A benefit of doing this is that Vue has an opportunity to parse it before the browser does
- This allows for developer experience improvements such as:
 - Self-closing tags
 - Easy enhancement path if needed

TECHNIQUE

v-bind="{ ... }"

v-on="{ ... }"

When working with props and/or events consider...



```
<VueMultiselect  
  v-bind:options="options"  
  v-bind:value="value"  
  v-bind:label="label"  
  v-on:change="parseSelection"  
  v-on:keyup="registerSelection"  
  v-on:mouseover="registerHover"  
>
```

When working with props and/or events consider...

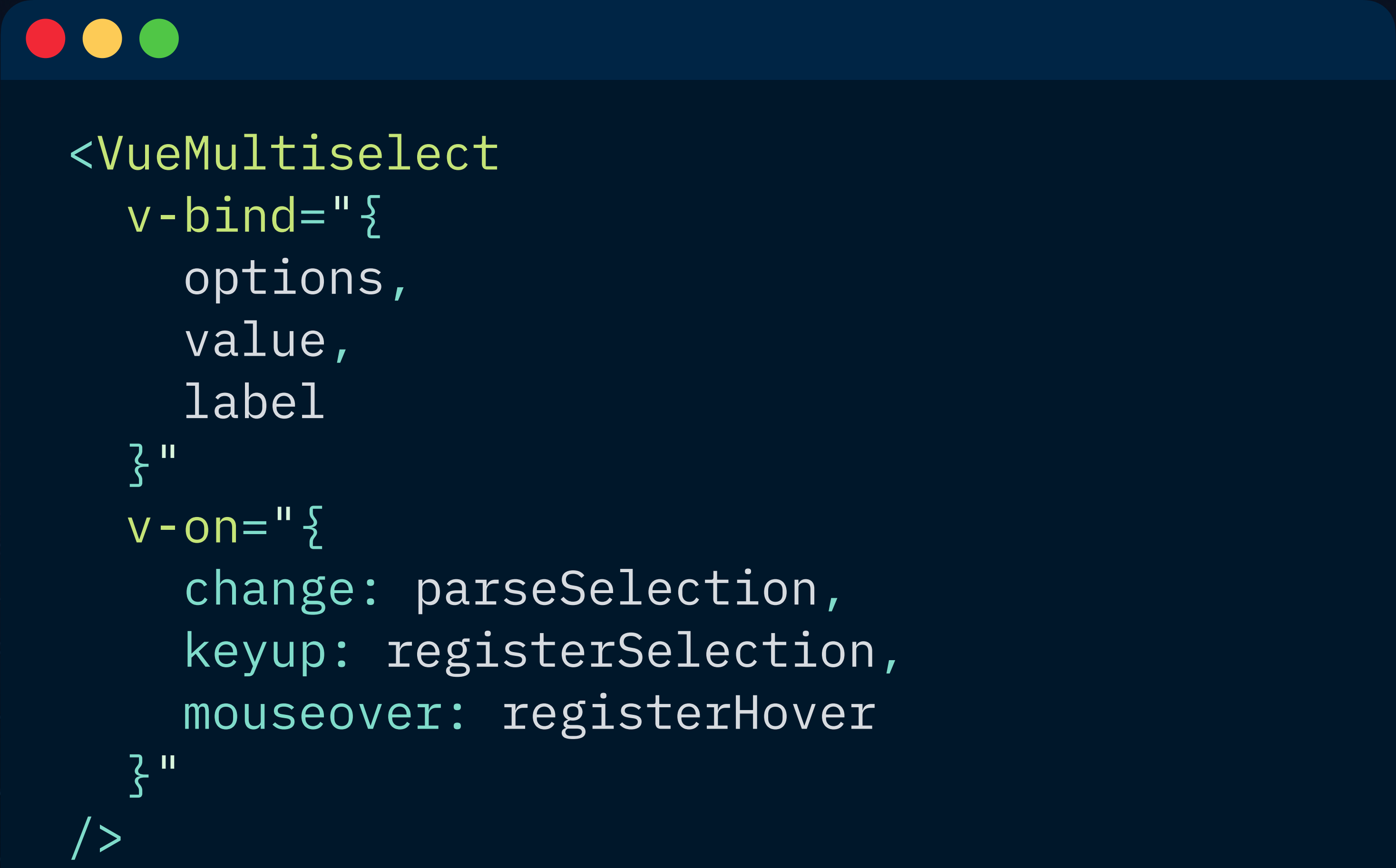


```
<VueMultiselect
  v-bind="{
    options: options,
    value: value,
    label: label
  }"
  v-on:change="parseSelection"
  v-on:keyup="registerSelection"
  v-on:mouseover="registerHover"
/>
```

When working with props and/or events consider...

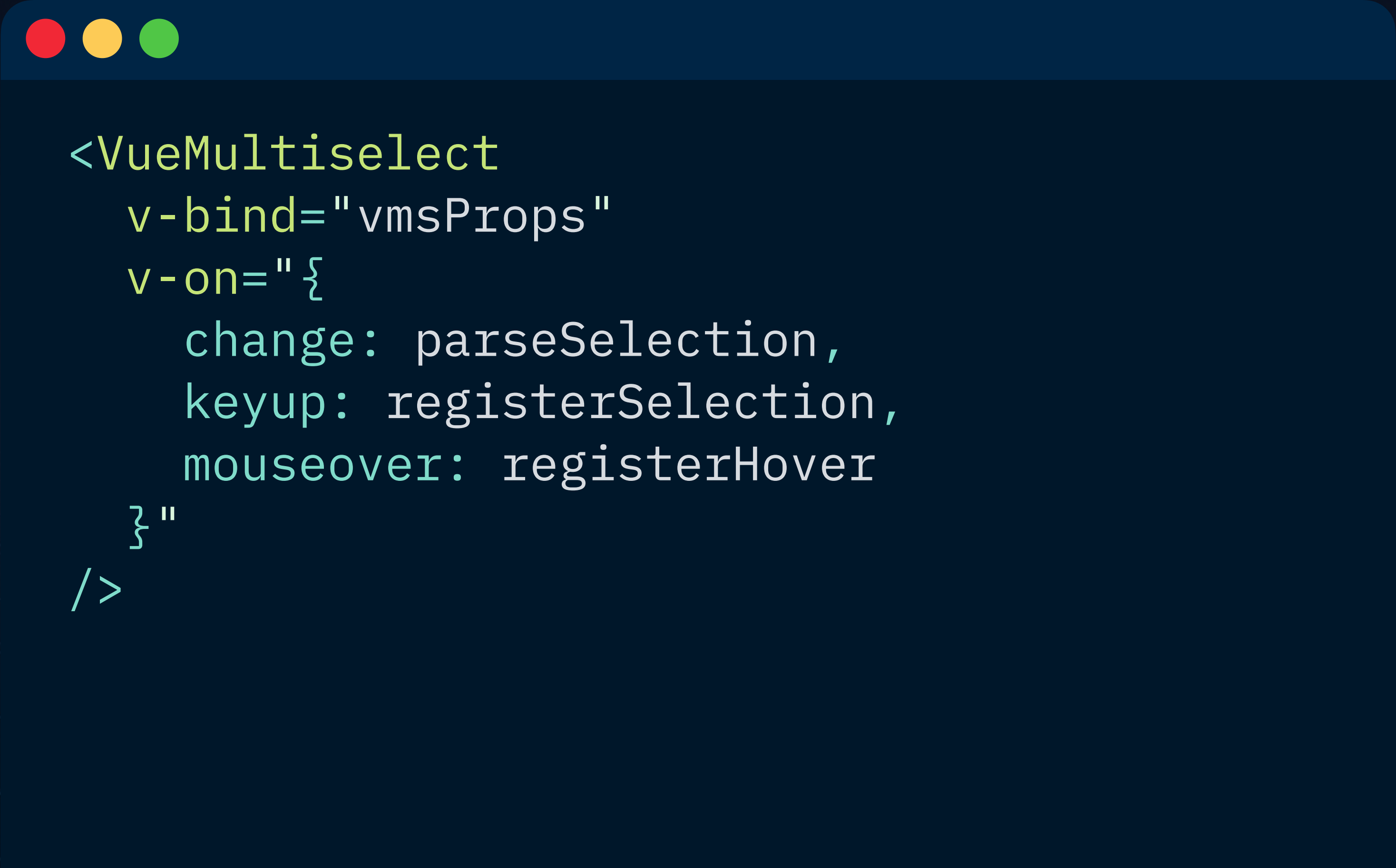
```
<VueMultiselect
  v-bind="{
    options: options,
    value: value,
    label: label
  }"
  v-on="{
    change: parseSelection,
    keyup: registerSelection,
    mouseover: registerHover
  }"
/>
```

When working with props and/or events consider...



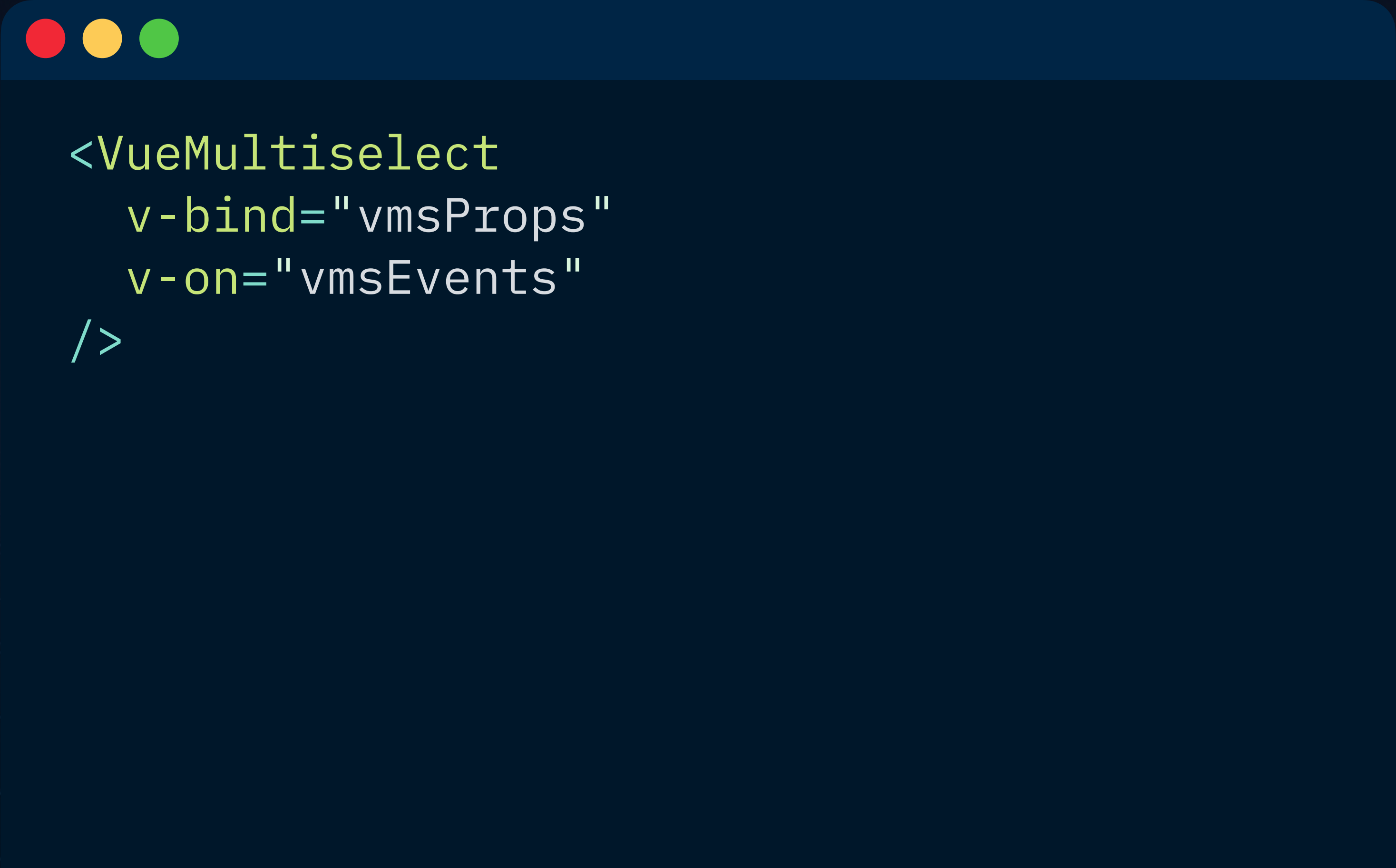
```
<VueMultiselect
  v-bind="{
    options,
    value,
    label
  }"
  v-on="{
    change: parseSelection,
    keyup: registerSelection,
    mouseover: registerHover
  }"
/>
```

When working with props and/or events consider...



```
<VueMultiselect  
  v-bind="vmsProps"  
  v-on="{  
    change: parseSelection,  
    keyup: registerSelection,  
    mouseover: registerHover  
  }"  
>
```

When working with props and/or events consider...



```
<VueMultiselect  
  v-bind="vmsProps"  
  v-on="vmsEvents"  
/>
```

CONVENTION

Don't use v-if with v-for.

CONVENTION

Don't use v-if with v-for.

```
<template>
  <ul>
    <li
      v-for="user in users"
      v-if="user.isActive"
      :key="user.id"
    >
      {{ user.name }}
    </li>
  </ul>
</template>
```



Bad

CONVENTION

Don't use v-if with v-for.

```
<template>
  <ul>
    <template v-for="user in users" :key="user.id">
      <li v-if="user.isActive">
        {{ user.name }}
      </li>
    </template>
  </ul>
</template>
```



CONVENTION

Don't use v-if with v-for.

```
<script setup>
import { ref, computed } from 'vue'
import { userData } from './userData.json'

const users = ref(userData)
const activeUsers = computed(() => {
  return users.value.filter(user => user.isActive)
})
</script>

<template>
  <ul>
    <li
      v-for="user in activeUsers"
      :key="user.id"
    >
      {{ user.name }}
    </li>
  </ul>
</template>
```



TECHNIQUE

Inspect complex data in the UI

TECHNIQUE

Inspect complex data in the UI



MyFirstComponent.vue

```
<script setup lang="ts">
import { ref } from 'vue'

const data = ref({
  postId: 1,
  id: '64456ca6-8927-46d6-9ee0-6afa99e508d2',
  name: 'Chidi Anagonye',
  email: 'myStomachHurts@thegoodplace.com',
  body: 'Principles aren't principles when you pick and choose
when you're gonna follow them.',
})
</script>

<template>
  <pre>{{ data }}</pre>
</template>
```



Questions?

LANGUAGES

CSS

CONVENTION

Limit global styles to App.vue

CONVENTION

Limit global styles to App.vue

1. Global styles should be documented at a single entry point.
2. App.vue is the simplest option since it's the global root component.
3. Minimizes accidental side effects.

CONVENTION

Scope all component styles.



MyRedText.vue

```
<template>
  <p class="red">
    This should be red!
  </p>
</template>
```

```
<style>
.red {
  color: red;
}
.bold {
  font-weight: bold;
}
</style>
```

TECHNIQUE

Scoped Styles



MyRedText.vue

```
<template>
  <p class="red">
    This should be red!
  </p>
</template>
```

```
<style>
.red {
  color: red;
}
.bold {
  font-weight: bold;
}
</style>
```



MyRedText.vue

```
<template>
  <p class="red">
    This should be red!
  </p>
</template>
```

```
<style scoped>
.red {
  color: red;
}
.bold {
  font-weight: bold;
}
</style>
```



MyRedText.vue

```
<template>
  <p class="red" data-v57s8>
    This should be red!
  </p>
</template>
```

```
<style>
.red[data-v57s8] {
  color: red;
}
.bold[data-v57s8] {
  font-weight: bold;
}
</style>
```

TECHNIQUE

CSS Modules



MyRedText.vue

```
<template>
  <p class="red">
    This should be red!
  </p>
</template>
```

```
<style>
.red {
  color: red;
}
.bold {
  font-weight: bold;
}
</style>
```



MyRedText.vue

```
<template>
  <p class="red">
    This should be red!
  </p>
</template>
```

```
<style module>
.red {
  color: red;
}
.bold {
  font-weight: bold;
}
</style>
```



MyRedText.vue

```
<template>
  <p :class="$style.red">
    This should be red!
  </p>
</template>
```

```
<style module>
.red {
  color: red;
}
.bold {
  font-weight: bold;
}
</style>
```



MyRedText.vue

```
<template>
  <p :class="$style.red">
    This should be red!
  </p>
</template>
```

```
<style>
.MyRedText_red_3fj4x {
  color: red;
}
.MyRedText_bold_8fn3s {
  font-weight: bold;
}
</style>
```



MyRedText.vue

```
<template>
  <p class="MyRedText_red_3fj4x">
    This should be red!
  </p>
</template>
```

```
<style>
.MyRedText_red_3fj4x {
  color: red;
}
.MyRedText_bold_8fn3s {
  font-weight: bold;
}
</style>
```

TECHNIQUE

CSS Modules Exports

TECHNIQUE

CSS Modules Exports



```
<template>
  <p>Grid Padding: {{ $style.gridPadding }}</p>
</template>
```

```
<style module>
:export {
  gridPadding: 1.5rem;
}
</style>
```

DISCUSSION

Scoped Styles vs CSS Modules

TECHNIQUE

State Driven CSS

TECHNIQUE

State Driven CSS

```
<template>
  <h1 class="title">Make my color dynamic!</h1>
</template>

<style scoped>
.title {
  color: red;
}
</style>
```



TECHNIQUE

State Driven CSS



```
<script>
export default {
  data: () => ({
    textColor: 'papayawhip'
  })
}
</script>

<template>
  <h1 class="title">Color me whatever you want!</h1>
  <input v-model="textColor" type="color" />
</template>

<style scoped>
.title {
  color: v-bind(textColor);
}
</style>
```



Questions?

LANGUAGES

JavaScript

DISCUSSION

JavaScript vs TypeScript

DISCUSSION

JavaScript vs TypeScript

- Majority of bugs encountered are **not** due to type violations
- TypeScript does **not** inherently guarantee type safety. It requires discipline
- Full type safety in a codebase can be a significant cost to a team in terms of productivity
- Most applications would benefit from better tests and code reviews

DISCUSSION

JavaScript vs TypeScript

- Progressive types can be added to a codebase with JSDoc comments.

```
/**
 * @param {Comment} comment
 * @returns {string}
 */
function formatCommentAuthor(comment) {
  return `${comment.name} <${comment.email}>`
}
```

DISCUSSION

JavaScript vs TypeScript

- At the end of day...
 - If the application is in Vue.js 2, probably not worth upgrading to TypeScript.
 - If it's a brand new Vue 3 project, it's recommended to use TypeScript.

CONVENTION

Prefer destructuring over multiple arguments

CONVENTION

Prefer destructuring over multiple arguments

```
updateUser (userList, index, value, isOnline) {  
  if (isOnline) {  
    userList[index] = value  
  } else {  
    this.removeUser(userList, index)  
  }  
}
```

✗ Not recommended

CONVENTION

Prefer destructuring over multiple arguments

```
updateUser ({ userList, index, value, isOnline }) {  
  if (isOnline) {  
    userList[index] = value  
  } else {  
    this.removeUser(userList, index)  
  }  
}
```

✓ Recommended



Questions?



CODE / EXPERIMENT



In the playground

- Create a component using the render function
- Experiment with different CSS techniques (scoped, modules, state driven, etc.)
- Try writing some TypeScript!

In your app

- Refactor a template to use the render function instead.
- Refactor a component to use CSS modules
- Setup TypeScript in your project and add it to a component.



SHORT BREAK

Be back at 10:45AM!



Components

BEST PRACTICE

Naming Components



Actual
programming



Debating for
30 minutes on
how to name a
variable

BEST PRACTICE

Naming Components

Avoid single word components

~~Header.vue~~

~~Button.vue~~

~~Container.vue~~

BEST PRACTICE

Naming Components

AppPrefixedName.vue / **Base**PrefixedName.vue

Reusable, globally registered UI components.

AppButton, AppModal, BaseDropdown, BaseInput

ThePrefixedName.vue

Single-instance components where only 1 can be active at the same time.

TheShoppingCart, TheSidebar, TheNavbar

BEST PRACTICE

Naming Components

Tightly coupled/related components

TodoList.vue

TodoListItem.vue

TodoListItemName.vue

1. Easy to spot relationship
2. Stays next to each other in the file tree
3. Obvious code smell if used in the wrong context

BEST PRACTICE

Naming Components

Page and route components

Pattern #1: **Suffix**

HomePage.vue

ProductDetailView.vue

Pattern #2: **URL Path**

index.vue

product/[detail-id].vue

TECHNIQUE

Props

NavItem.vue

```
<script>
  export default {
    props: ['label']
  }
</script>

<template>
  <li class="nav-item">
    <a :href="`/${label}`">
      {{ label }}
    </a>
  </li>
</template>
```

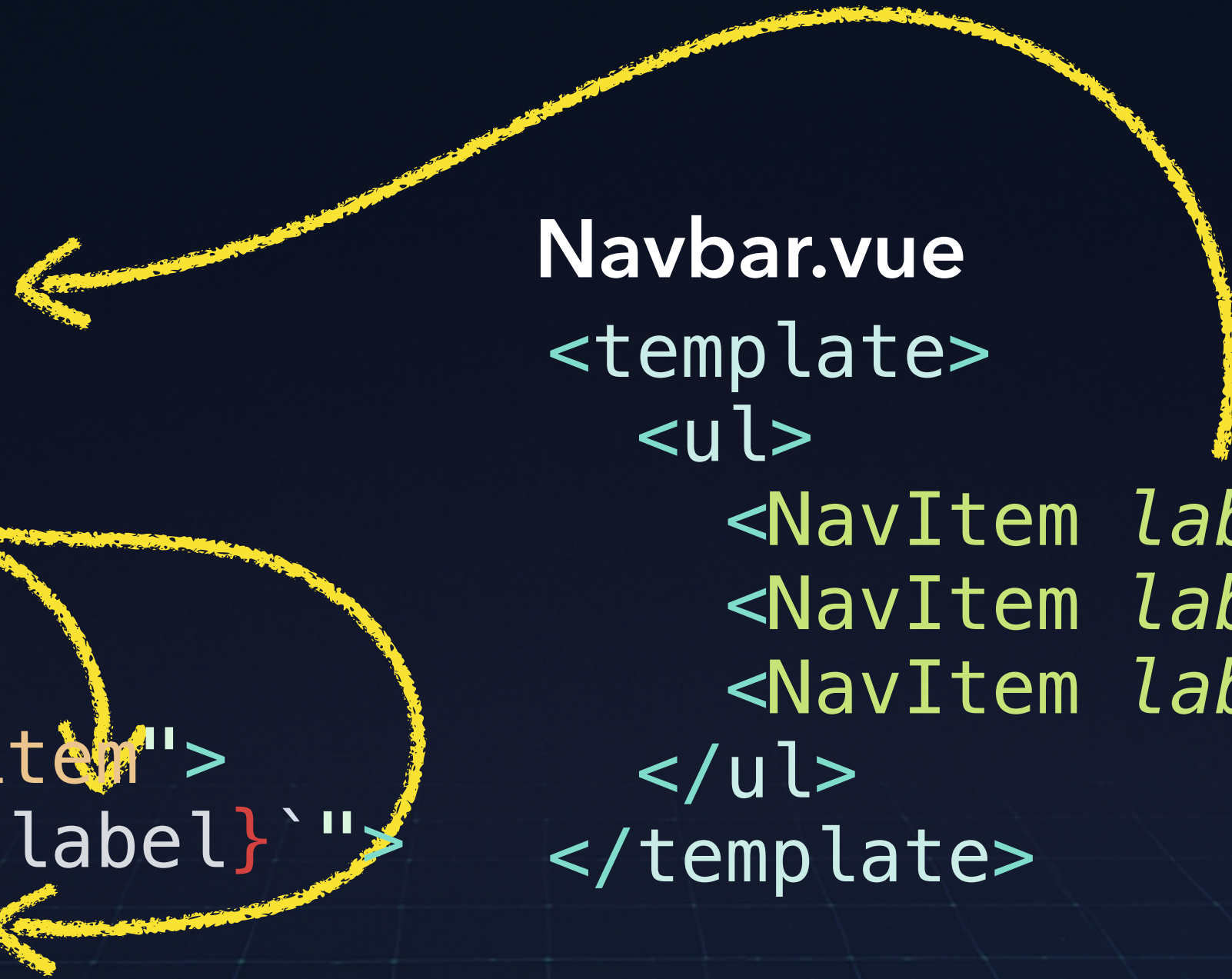
NavItem.vue

```
<script>
export default {
  props: ['label']
}
</script>

<template>
  <li class="nav-item">
    <a :href="'`/${label}`'">
      {{ label }}
    </a>
  </li>
</template>
```

Navbar.vue

```
<template>
  <ul>
    <NavItem label="Home" />
    <NavItem label="About" />
    <NavItem label="Contact" />
  </ul>
</template>
```



NavItem.vue

```
<script>
export default {
  props: ['label']
}
</script>

<template>
  <li class="nav-item">
    <a :href="'`/${label}`'">
      {{ label }}
    </a>
  </li>
</template>
```



Navbar.vue

```
<template>
  <ul>
    <NavItem label="Home" />
    <NavItem label="About" />
    <NavItem label="Contact" />
  </ul>
</template>
```

NavItem.vue

```
<script>  
export default {  
  props: ['label']  
}  
</script>
```

NavItem.vue

```
<script>
export default {
  props: {
    label: {
      type: String,
      required: true,
      default: 'Home'
    }
  }
}
</script>
```

NavItem.vue

```
<script>
export default {
  props: {
    label: {
      type: String,
      default: 'Home'
    }
  }
}
</script>
```

NavItem.vue

```
<script>
export default {
  props: {
    label: {
      type: String,
      default: 'Home',
      validator: (value) => {
        return ['Home', 'About'].indexOf(value) !== -1
      }
    }
  }
}
</script>
```

NavItem.vue

```
<script setup lang="ts">
export default {
  props: {
    label: {
      type: String,
      default: 'Home',
      validator: (value) => {
        return ['Home', 'About'].indexOf(value) !== -1
      }
    }
  }
}
</script>
```

NavItem.vue

```
<script setup lang="ts">
type Label = 'Home' | 'About'

export default {
  props: {
    label: {
      type: String,
      default: 'Home',
    }
  }
}
</script>
```

NavItem.vue

```
<script setup lang="ts">
type Label = 'Home' | 'About'

const { label } = defineProps<{
  label?: Label
}>()

export default {
  default: 'Home',
}
</script>
```

NavItem.vue

```
<script setup lang="ts">  
type Label = 'Home' | 'About'
```

```
const { label = 'Home' } = defineProps<{  
  label?: Label  
</script>
```

CONVENTION

Naming Methods

CONVENTION

Naming Methods

1. Use descriptive names

✗ `onInput`

✗ `handleClick`

✓ `updateUserName`

2. Don't assume where it will be called

```
updateUserName($event) {  
  this.user.name = $event.target.value  
}
```

✗ Wrong

```
updateUserName (newName) {  
  this.user.name = newName  
}
```

✓ Correct

CONVENTION

SFC Code Block Order



```
<template>  
  <h1>Hello VueConfUS 2026!</h1>  
</template>
```

```
<script setup lang="ts">  
  // My Component Logic  
</script>
```

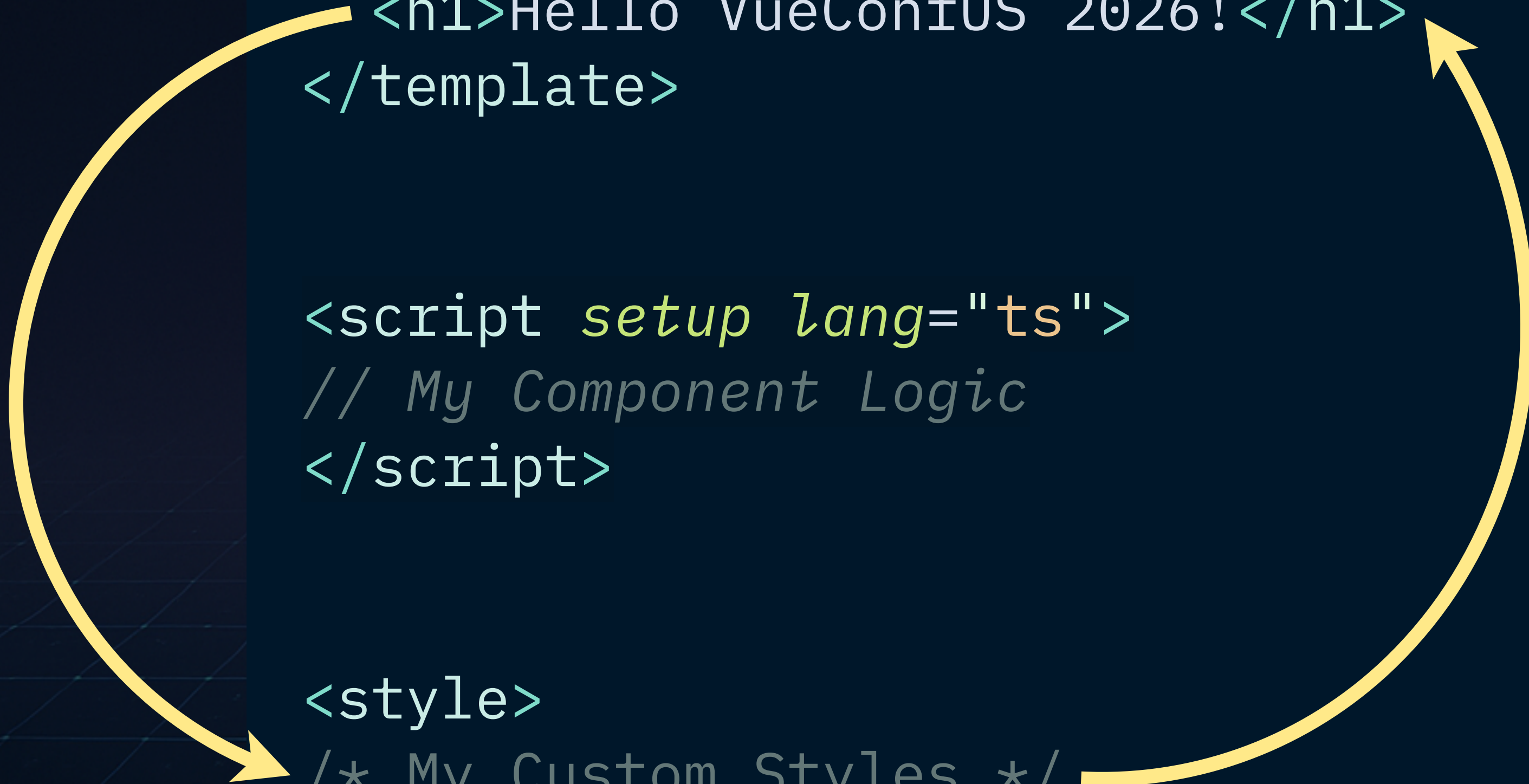
```
<style>  
  /* My Custom Styles */  
</style>
```



```
<template>  
  <h1>Hello VueConfUS 2026!</h1>  
</template>
```

```
<script setup lang="ts">  
  // My Component Logic  
</script>
```

```
<style>  
  /* My Custom Styles */  
</style>
```





```
<template>
```

```
<h1>Hello VueConfUS 2026!</h1>
```

```
</template>
```

```
<script setup lang="ts">
```

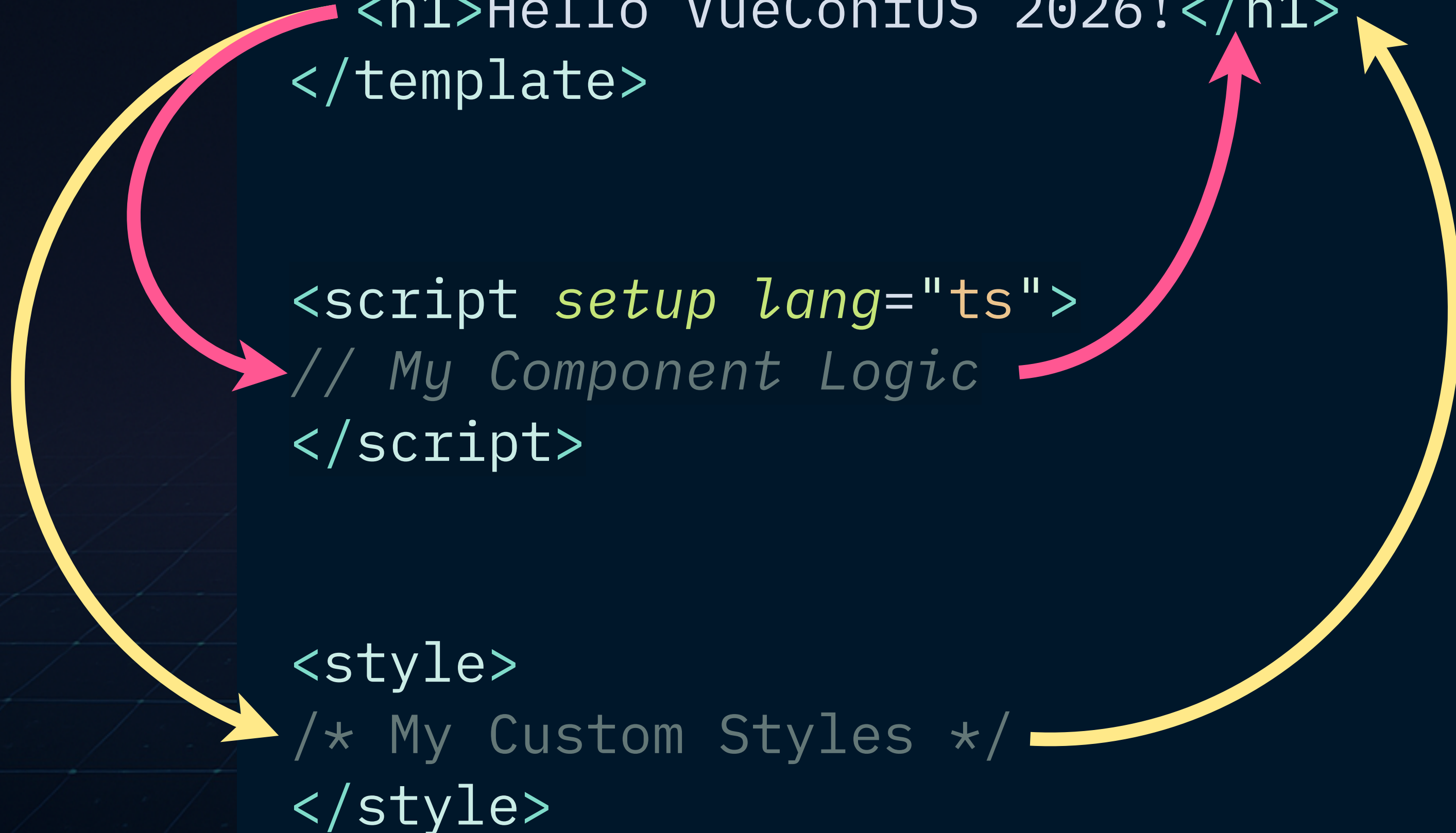
```
// My Component Logic
```

```
</script>
```

```
<style>
```

```
/* My Custom Styles */
```

```
</style>
```





```
<template>  
  <h1>Hello VueConfUS 2026!</h1>  
</template>
```

```
<script setup lang="ts">  
// My Component Logic  
</script>
```

```
<style>  
/* My Custom Styles */  
</style>
```



```
<script setup lang="ts">  
  // My Component Logic  
</script>
```

```
<template>  
  <h1>Hello VueConfUS 2026!</h1>  
</template>
```

```
<style>  
  /* My Custom Styles */  
</style>
```



```
<script setup lang="ts">
```

```
// My Component Logic
```

```
</script>
```

```
<template>
```

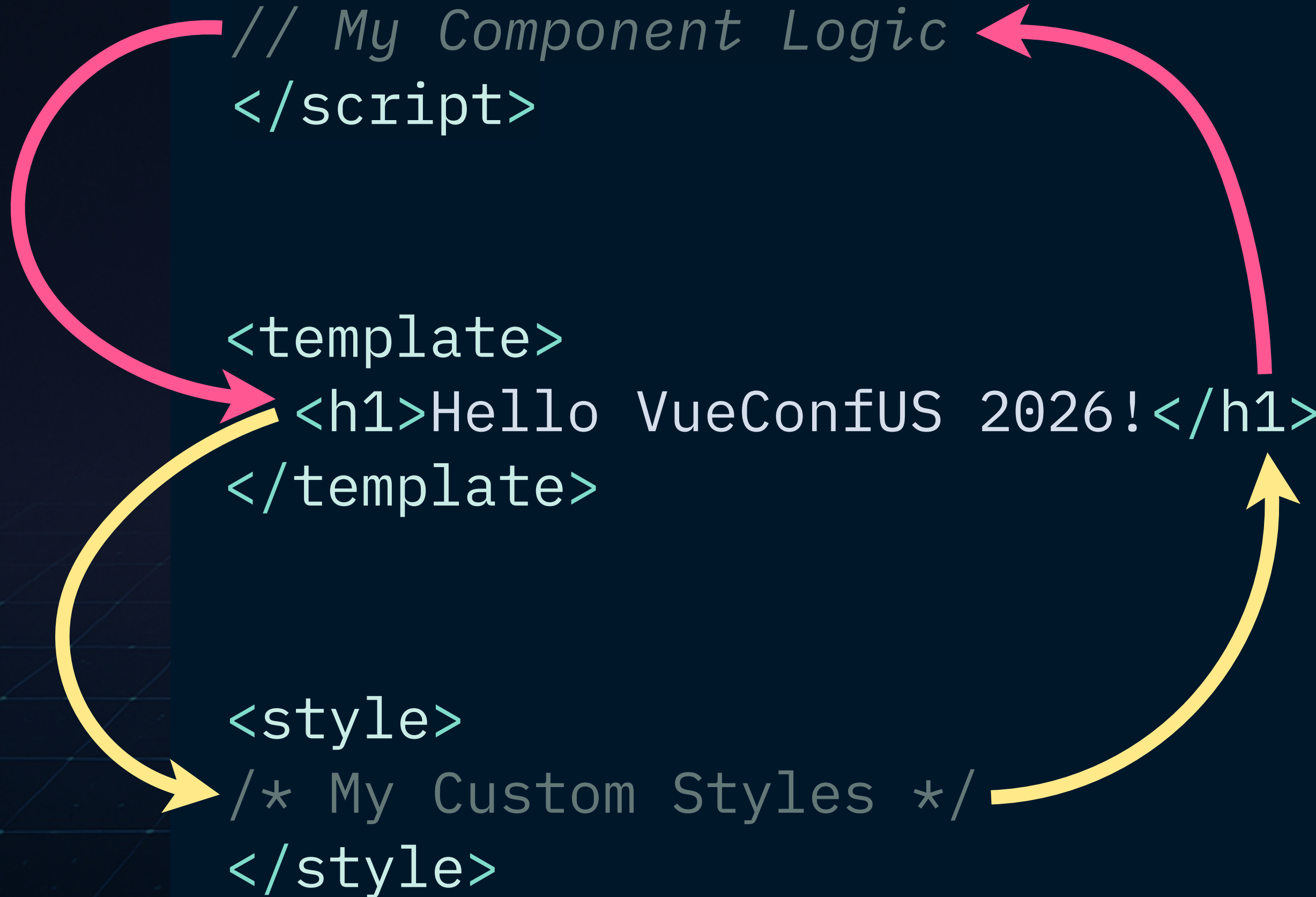
```
<h1>Hello VueConfUS 2026!</h1>
```

```
</template>
```

```
<style>
```

```
/* My Custom Styles */
```

```
</style>
```



CONVENTION

Component File Organization

Nested Structure

```
src/  
└── components/  
    ├── Dashboard/  
    │   ├── tests/  
    │   ├── Dashboard.vue  
    │   └── Header.vue  
    ├── Login/  
    │   ├── tests/  
    │   ├── Form.vue  
    │   ├── Header.vue  
    │   └── Login.vue  
    ├── tests/  
    └── Header.vue
```

Nested Structure

```
src/  
├── components/  
│   ├── Dashboard/  
│   │   ├── tests/  
│   │   │   ├── Dashboard.unit.ts  
│   │   │   └── Header.unit.ts  
│   │   ├── Dashboard.vue  
│   │   └── Header.vue  
│   ├── Login/  
│   │   ├── tests/  
│   │   │   ├── Header.unit.ts  
│   │   │   └── Login.unit.ts  
│   │   ├── Form.vue  
│   │   ├── Header.vue  
│   │   └── Login.vue  
│   ├── tests/  
│   │   └── Header.unit.ts  
│   └── Header.vue
```

Better Nested Structure

```
src/  
├── components/  
│   ├── Dashboard/  
│   │   ├── tests/  
│   │   │   ├── Dashboard.unit.ts  
│   │   │   └── Header.unit.ts  
│   │   ├── Dashboard.vue  
│   │   └── Header.vue  
│   ├── Login/  
│   │   ├── tests/  
│   │   │   ├── Header.unit.ts  
│   │   │   └── Login.unit.ts  
│   │   ├── Form.vue  
│   │   ├── Header.vue  
│   │   └── Login.vue  
│   ├── tests/  
│   │   └── Header.unit.ts  
│   └── Header.vue
```

Better Nested Structure

```
src/  
└── components/  
    ├── Dashboard/  
    │   ├── Dashboard.vue  
    │   ├── Dashboard.unit.ts  
    │   ├── Header.vue  
    │   └── Header.unit.ts  
    ├── Login/  
    │   ├── tests/  
    │   │   ├── Header.unit.ts  
    │   │   └── Login.unit.ts  
    │   ├── Form.vue  
    │   ├── Header.vue  
    │   └── Login.vue  
    ├── tests/  
    │   └── Header.unit.ts  
    └── Header.vue
```

Better Nested Structure

```
src/  
└── components/  
    ├── Dashboard/  
    │   ├── Dashboard.vue  
    │   ├── Dashboard.unit.ts  
    │   ├── Header.vue  
    │   └── Header.unit.ts  
    ├── Login/  
    │   ├── Form.vue  
    │   ├── Header.vue  
    │   ├── Header.unit.ts  
    │   ├── Login.vue  
    │   └── Login.unit.ts  
    ├── tests/  
    │   └── Header.unit.ts  
    └── Header.vue
```

Better Nested Structure

```
src/  
├── components/  
│   ├── Dashboard/  
│   │   ├── Dashboard.vue  
│   │   ├── Dashboard.unit.ts  
│   │   ├── Header.vue  
│   │   └── Header.unit.ts  
│   ├── Login/  
│   │   ├── Form.vue  
│   │   ├── Form.unit.ts  
│   │   ├── Header.vue  
│   │   ├── Header.unit.ts  
│   │   ├── Login.vue  
│   │   └── Login.unit.ts  
│   ├── tests/  
│   │   └── Header.unit.ts  
└── Header.vue
```

Better Nested Structure

```
src/  
└── components/  
    ├── Dashboard/  
    │   ├── Dashboard.vue  
    │   ├── Dashboard.unit.ts  
    │   ├── Header.vue  
    │   └── Header.unit.ts  
    ├── Login/  
    │   ├── Form.vue  
    │   ├── Form.unit.ts  
    │   ├── Header.vue  
    │   ├── Header.unit.ts  
    │   ├── Login.vue  
    │   └── Login.unit.ts  
    ├── Header.vue  
    └── Header.unit.ts
```

Flat Structure

```
src/  
└── components/  
    ├── Dashboard/  
    │   ├── Dashboard.vue  
    │   ├── Dashboard.unit.ts  
    │   ├── Header.vue  
    │   └── Header.unit.ts  
    ├── Login/  
    │   ├── Form.vue  
    │   ├── Form.unit.ts  
    │   ├── Header.vue  
    │   ├── Header.unit.ts  
    │   ├── Login.vue  
    │   └── Login.unit.ts  
    ├── Header.vue  
    └── Header.unit.ts
```

Flat Structure

```
src/  
└─ components/  
    └─ Dashboard.unit.ts  
    └─ Dashboard.vue  
    └─ DashboardHeader.unit.ts  
    └─ DashboardHeader.vue  
    └─ Login/  
        └─ Form.vue  
        └─ Form.unit.ts  
        └─ Header.vue  
        └─ Header.unit.ts  
        └─ Login.vue  
        └─ Login.unit.ts  
    └─ Header.vue  
    └─ Header.unit.ts
```

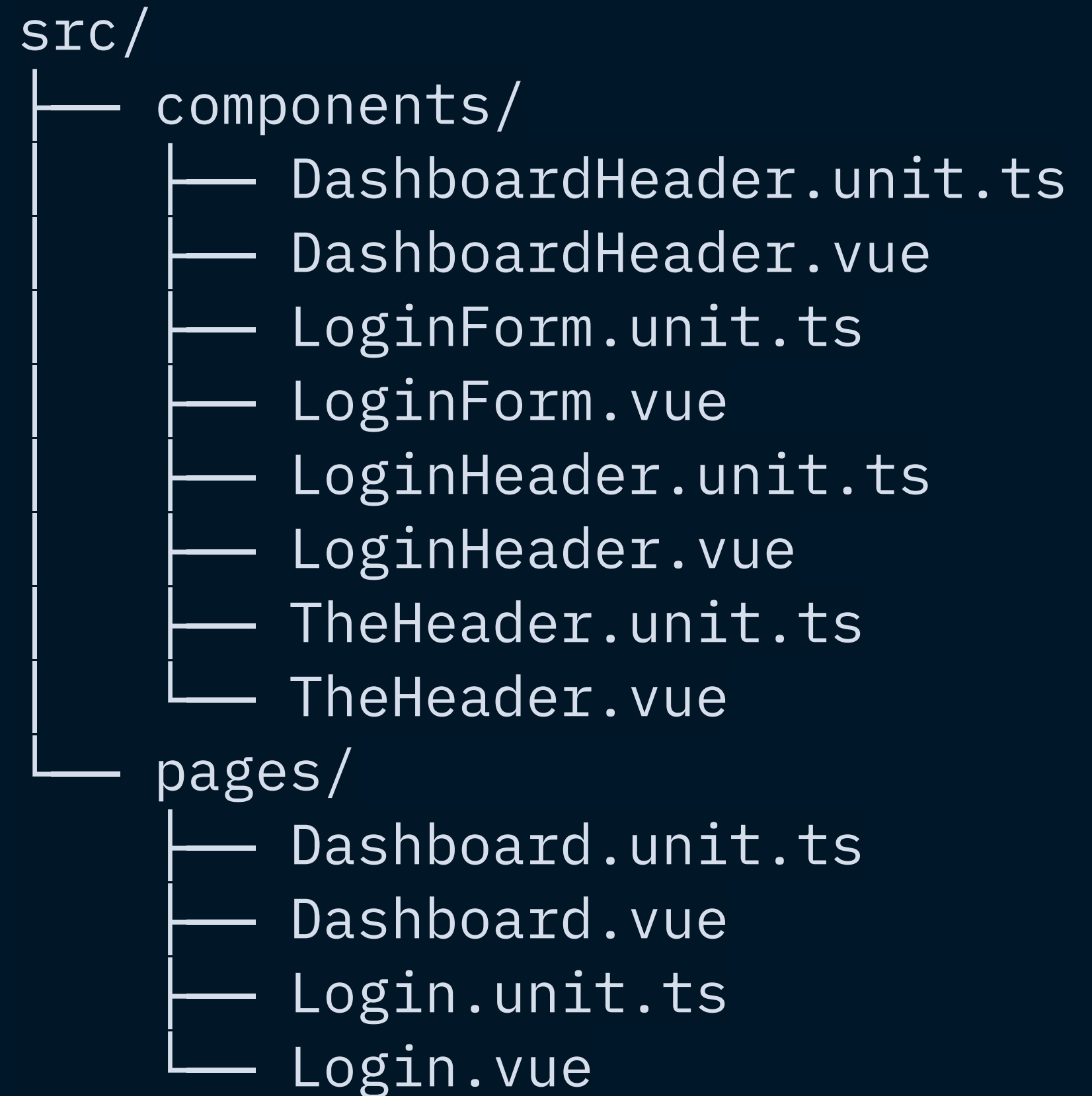
Flat Structure

```
src/  
└─ components/  
    └─ Dashboard.unit.ts  
    └─ Dashboard.vue  
    └─ DashboardHeader.unit.ts  
    └─ DashboardHeader.vue  
    └─ Login.unit.ts  
    └─ Login.vue  
    └─ LoginForm.unit.ts  
    └─ LoginForm.vue  
    └─ LoginHeader.unit.ts  
    └─ LoginHeader.vue  
    └─ Header.vue  
    └─ Header.unit.ts
```

Flat Structure

```
src/  
└─ components/  
    └─ Dashboard.unit.ts  
    └─ Dashboard.vue  
    └─ DashboardHeader.unit.ts  
    └─ DashboardHeader.vue  
    └─ Login.unit.ts  
    └─ Login.vue  
    └─ LoginForm.unit.ts  
    └─ LoginForm.vue  
    └─ LoginHeader.unit.ts  
    └─ LoginHeader.vue  
    └─ TheHeader.unit.ts  
    └─ TheHeader.vue
```

Better Flat Structure



```
src/  
├── components/  
│   ├── DashboardHeader.unit.ts  
│   ├── DashboardHeader.vue  
│   ├── LoginForm.unit.ts  
│   ├── LoginForm.vue  
│   ├── LoginHeader.unit.ts  
│   ├── LoginHeader.vue  
│   ├── TheHeader.unit.ts  
│   └── TheHeader.vue  
└── pages/  
    ├── Dashboard.unit.ts  
    ├── Dashboard.vue  
    ├── Login.unit.ts  
    └── Login.vue
```

Better Flat Structure

```
src/  
├── components/  
│   ├── DashboardHeader.unit.ts  
│   ├── DashboardHeader.vue  
│   ├── LoginForm.unit.ts  
│   ├── LoginForm.vue  
│   ├── LoginHeader.unit.ts  
│   ├── LoginHeader.vue  
│   ├── TheHeader.unit.ts  
│   └── TheHeader.vue  
└── pages/  
    ├── dashboard.unit.ts  
    ├── dashboard.vue  
    ├── login.unit.ts  
    └── login.vue
```

```
src/
├── components/
│   ├── Dashboard/
│   │   ├── tests/
│   │   │   ├── Dashboard.unit.ts
│   │   │   └── Header.unit.ts
│   │   ├── Dashboard.vue
│   │   └── Header.vue
│   ├── Login/
│   │   ├── tests/
│   │   │   ├── Header.unit.ts
│   │   │   └── Login.unit.ts
│   │   ├── Form.vue
│   │   ├── Header.vue
│   │   └── Login.vue
│   ├── tests/
│   │   └── Header.unit.ts
│   └── Header.vue
```

VS

```
src/
├── components/
│   ├── DashboardHeader.unit.ts
│   ├── DashboardHeader.vue
│   ├── LoginForm.unit.ts
│   ├── LoginForm.vue
│   ├── LoginHeader.unit.ts
│   ├── LoginHeader.vue
│   ├── TheHeader.unit.ts
│   └── TheHeader.vue
└── pages/
    ├── dashboard.unit.ts
    ├── dashboard.vue
    ├── login.unit.ts
    └── login.vue
```

CONVENTION

Component File Organization

Flat makes refactoring easier

No need to update imports if components move

Flat makes finding files easier

Folders often leads to lazily named files because they don't have to be unique

Flat exposes broken conventions

It's easy to spot when a file is missing a companion file(s)

TECHNIQUE

Namespaced Components



```
// @/App.vue
<script setup lang="ts">
import TodoList from '@components/todo/TodoList.vue'
import TodoItem from '@components/todo/TodoItem.vue'
</script>

<template>
  <div>
    <TodoList>
      <TodoItem v-for="i in 5" :key="i"> Todo {{ i }} </TodoItem>
    </TodoList>
  </div>
</template>
```



```
// @/App.vue
<script setup lang="ts">
import * as Todo from '@components/todo'
</script>

<template>
  <div>
    <Todo.List>
      <Todo.Item v-for="i in 5" :key="i"> Todo {{ i }} </Todo.Item>
    </Todo.List>
  </div>
</template>
```



```
// @/components/index.ts
```

```
export { default as List } from './TodoList.vue'
```

```
export { default as Item } from './TodoItem.vue'
```

```
// @/App.vue
```

```
<script setup lang="ts">
```

```
import * as Todo from '@components/todo'
```

```
</script>
```

```
<template>
```

```
  <div>
```

```
    <Todo.List>
```

```
      <Todo.Item v-for="i in 5" :key="i"> Todo {{ i }} </Todo.Item>
```

```
    </Todo.List>
```

```
  </div>
```

```
</template>
```

CONVENTION

Auto import components



```
<script setup lang="ts">
import { computed, ref } from 'vue'
import { useProjects } from '@/composables/useProjects'
import AppButton from '@/components/AppButton.vue'
import AppCard from '@/components/AppCard.vue'
import AppIcon from '@/components/AppIcon.vue'
import AppInput from '@/components/AppInput.vue'
import AppSelect from '@/components/AppSelect.vue'
import AppTable from '@/components/AppTable.vue'
import AppTableColumn from '@/components/AppTableColumn.vue'

const statusOptions = ['Active', 'Paused', 'Archived']
const search = ref('')
const selectedStatus = ref('Active')
const { projects } = useProjects()

const filteredProjects = computed(() => {
  return projects.value.filter((project) => {
    return project.name.includes(search.value) && project.status === selectedStatus.value
  })
})
</script>
```

```
<template>
  <AppCard>
    <AppInput v-model="search" label="Search projects" />

    <AppSelect v-model="selectedStatus" label="Status" :options="statusOptions" />

    <AppTable :items="filteredProjects">
      <AppTableColumn field="name" label="Name" />
      <AppTableColumn field="owner" label="Owner" />
      <AppTableColumn field="status" label="Status" />
    </AppTable>

    <AppButton>
      <AppIcon name="plus" />
      New project
    </AppButton>
  </AppCard>
</template>
```



```
<script setup lang="ts">
import { computed, ref } from 'vue'
import { useProjects } from '@composables/useProjects'

const statusOptions = ['Active', 'Paused', 'Archived']
const search = ref('')
const selectedStatus = ref('Active')
const { projects } = useProjects()

const filteredProjects = computed(() => {
  return projects.value.filter((project) => {
    return project.name.includes(search.value) && project.status === selectedStatus.value
  })
})
</script>

<template>
  <AppCard>
    <AppInput v-model="search" label="Search projects" />

    <AppSelect v-model="selectedStatus" label="Status" :options="statusOptions" />

    <AppTable :items="filteredProjects">
      <AppTableColumn field="name" label="Name" />
      <AppTableColumn field="owner" label="Owner" />
      <AppTableColumn field="status" label="Status" />
    </AppTable>

    <AppButton>
      <AppIcon name="plus" />
      New project
    </AppButton>
  </AppCard>
</template>
```

CONVENTION

Auto import components

No more multi-line component import statements...

```
import BaseButton from '@components/BaseButton.vue'  
import BaseIcon from '@components/BaseIcon.vue'  
import BaseInput from '@components/BaseInput.vue'
```

Do this in a performative way with:

<https://github.com/antfu/unplugin-vue-components>

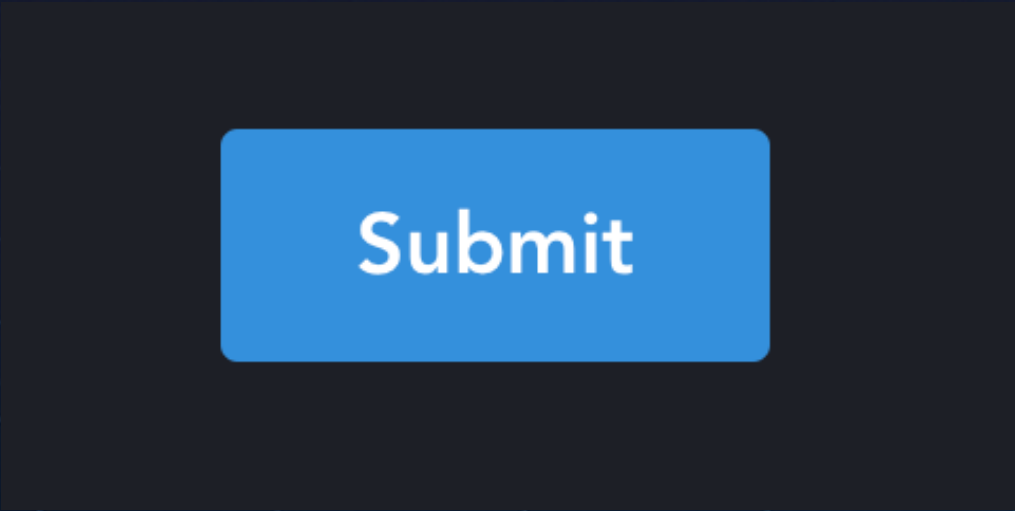


Questions?

**Let's do a coding
thought experiment !**

Task 1

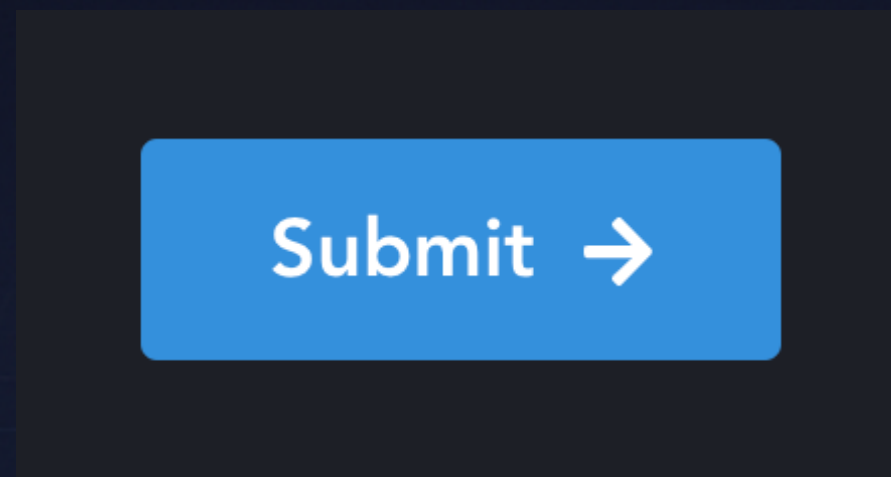
Create a button component that can display text specified in the parent component



Submit

Task 2

Allow the button to display an icon of choice
on the right side of the text



```
<AppIcon icon="arrow-right" class="ml-3" />
```

This is the code responsible for displaying an arrow.

Task 3

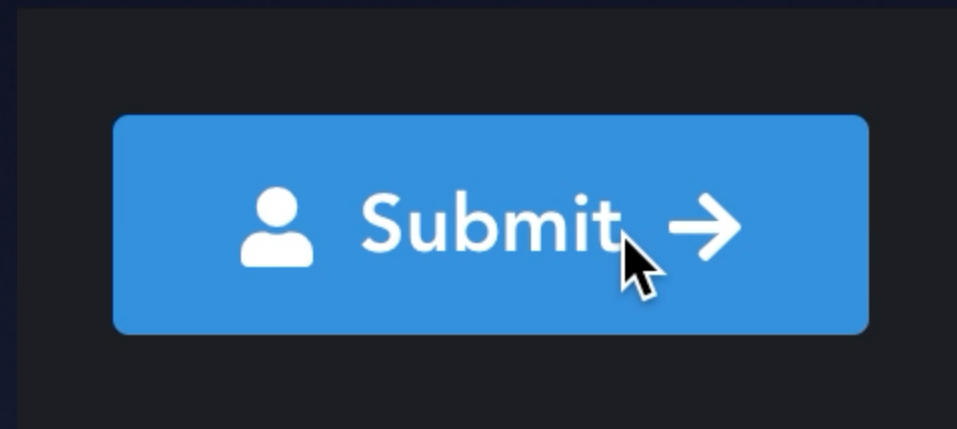
Make it possible to have icons on either side or even both sides



 Submit →

Task 4

Make it possible to replace the content with a loading spinner

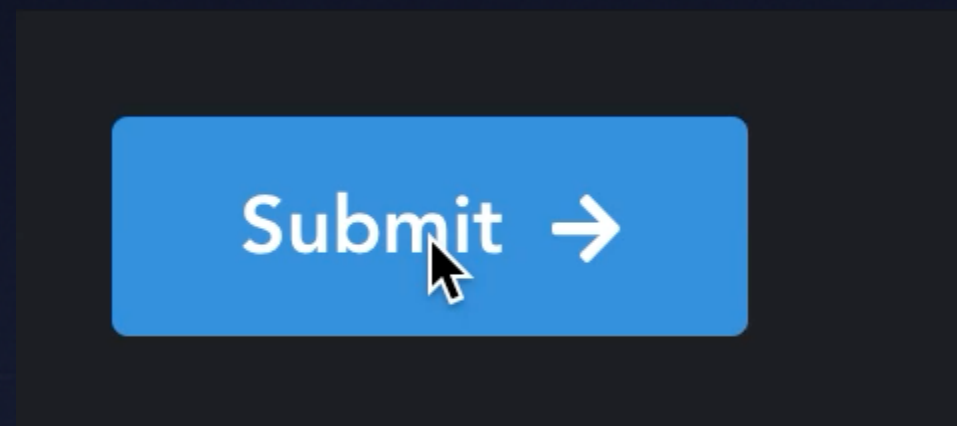


```
<PulseLoader color="#fff" size="12px" />
```

This is the code responsible for displaying a spinner.

Task 5

Make it possible to replace an icon with a loading spinner



Possible solution...



```
<template>
  <button type="button" class="nice-button">
    {{ text }}
  </button>
</template>
```

```
<script setup lang="ts">
defineProps(['text'])
</script>
```



```
<template>
  <button type="button" class="nice-button">
    <PulseLoader v-if="isLoading" color="#fff" size="12px" />
    <template v-else>
      <template v-if="iconLeftName">
        <PulseLoader v-if="isLoadingLeft" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconLeftName" />
      </template>
      {{ text }}
      <template v-if="iconRightName">
        <PulseLoader v-if="isLoadingRight" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconRightName" />
      </template>
    </template>
  </button>
</template>
```

```
<script setup lang="ts">
defineProps(['text', 'iconLeftName', 'iconRightName', 'isLoading',
'isLoadingLeft', 'isLoadingRight'])
</script>
```



```
<template>
  <button typ
    <PulseLoa
    <template
      <templa
        <Puls
        <AppI
      </templ
    {{ text
  <templa
    <Puls
    <AppI
  </templ
</templat
</button>
</template>
```

```
<script setup
defineProps([
'isLoadingLef
</script>
```



OMG
PROPS EVERYWHERE!



```
<template>
  <button type="button" class="nice-button">
    <PulseLoader v-if="isLoading" color="#fff" size="12px" />
    <template v-else>
      <template v-if="iconLeftName">
        <PulseLoader v-if="isLoadingLeft" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconLeftName" />
      </template>
      {{ text }}
      <template v-if="iconRightName">
        <PulseLoader v-if="isLoadingRight" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconRightName" />
      </template>
    </template>
  </button>
</template>
```

Let's call it the **props-based** solution

```
<script setup lang="ts">
defineProps(['text', 'iconLeftName', 'iconRightName', 'isLoading',
'isLoadingLeft', 'isLoadingRight'])
</script>
```



```
<template>
  <button type="button" class="nice-button">
    <PulseLoader v-if="isLoading" color="#fff" size="12px" />
    <template v-else>
      <template v-if="iconLeftName">
        <PulseLoader v-if="isLoadingLeft" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconLeftName" />
      </template>
      {{ text }}
      <template v-if="iconRightName">
        <PulseLoader v-if="isLoadingRight" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconRightName" />
      </template>
    </template>
  </button>
</template>

<script setup lang="ts">
defineProps(['text', 'iconLeftName', 'iconRightName', 'isLoading',
'isLoadingLeft', 'isLoadingRight'])
</script>
```

props-based solution

Is it wrong?

props-based solution

Is it wrong?

No.

It does the job.

props-based solution

Is it good, then?

props-based solution

Is it good, then?

Not exactly.

props-based solution

Problems

props-based solution

Problems

- New requirements increase complexity
- Multiple responsibilities
- Lots of conditionals in the template
- Low flexibility
- Hard to maintain

props-based solution

Is it good, then?

Not exactly.

Is there a better another alternative?



Is there a better another alternative?

Recommended solution

```
<template>  
  <button type="button" class="nice-button">  
    <slot />  
  </button>  
</template>
```

TECHNIQUE

Slots



```
<template>
  <button type="button" class="nice-button">
    <PulseLoader v-if="isLoading" color="#fff" size="12px" />
    <template v-else>
      <template v-if="iconLeftName">
        <PulseLoader v-if="isLoadingLeft" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconLeftName" />
      </template>
      {{ text }}
      <template v-if="iconRightName">
        <PulseLoader v-if="isLoadingRight" color="#fff" size="6px" />
        <AppIcon v-else :icon="iconRightName" />
      </template>
    </template>
  </button>
</template>
```

```
<script setup lang="ts">
defineProps(['text', 'iconLeftName', 'iconRightName', 'isLoading',
'isLoadingLeft', 'isLoadingRight'])
</script>
```



BaseButton.vue

```
<template>
  <button type="button" class="nice-button">
    <slot />
  </button>
</template>
```



BaseButton.vue

```
<template>
  <button type="button" class="nice-button">
    <slot />
  </button>
</template>
```

App.vue

```
<template>
  <BaseButton>
    Submit
  </BaseButton>
</template>
```



BaseButton.vue

```
<template>
  <button type="button" class="nice-button">
    <slot />
  </button>
</template>
```

App.vue

```
<template>
  <BaseButton>
    Submit
    <PulseLoader v-if="isLoading" color="#fff" size="6px" />
    <AppIcon v-else icon="arrow-right" />
  </BaseButton>
</template>
```

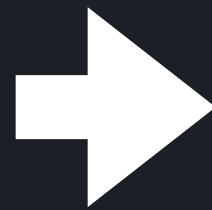
PROBLEM

What if you want to define multiple slots?

Default Slot

NavigationLink.vue

```
<a  
  :href="url"  
  class="nav-link"  
>  
  <slot></slot>  
</a>
```



```
<navigation-link url="/profile">  
  <span class="fa fa-user"/>  
  Your Profile  
</navigation-link>
```

Named Slots

BaseLayout.vue

```
<div class="container">
  <header>
    <slot name="header"></slot>
  </header>
  <main>
    <slot></slot>
  </main>
  <footer>
    <slot name="footer"></slot>
  </footer>
</div>
```



```
<base-layout>
  <template v-slot:header>
    <h1>Here might be a page title</h1>
  </template>

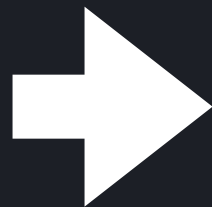
  <p>A paragraph for the main content.</p>
  <p>And another one.</p>

  <template v-slot:footer>
    Here's some contact info
  </template>
</base-layout>
```

Named Slots

BaseLayout.vue

```
<div class="container">
  <header>
    <slot name="header"></slot>
  </header>
  <main>
    <slot></slot>
  </main>
  <footer>
    <slot name="footer"></slot>
  </footer>
</div>
```



```
<base-layout>
  <template #header>
    <h1>Here might be a page title</h1>
  </template>

  <p>A paragraph for the main content.</p>
  <p>And another one.</p>

  <template #footer>
    Here's some contact info
  </template>
</base-layout>
```

Dynamic Slot Names

```
<base-layout>  
  <template v-slot:[dynamicSlotName]>  
    ...  
  </template>  
</base-layout>
```

PROBLEM

How do you access component data from the slot?

Scoped Slots (i.e., Slot Props)

```
// todo-list.vue
```

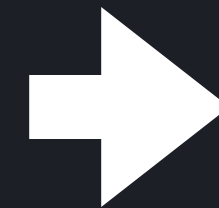
```
<ul>
  <li v-for="todo in todos" :key="todo.id">
    <slot name="header" :header="todo.header" >
      <!-- Fallback content -->
      {{ todo.text }}
    </slot>
    <slot :todo="todo" :newData="newData" >
      <!-- Fallback content -->
      {{ todo.text }}
    </slot>
  </li>
</ul>
```



```
<todo-list :todos="todos">
  <template v-slot="slotProps">
    {{ slotProps.todo }}
  </template>
  <template v-slot:header="slotProps">
    {{ slotProps.header }}
  </template>
</todo-list>
```

Destructuring slot-scope

```
<todo-list :todos="todos">
  <template v-slot="slotProps">
    <AppIcon
      v-if="slotProps.todo.completed"
      icon="checked"
    />
    {{ slotProps.todo.text }}
  </template>
</todo-list>
```



```
<todo-list :todos="todos">
  <template v-slot="{ todo }">
    <AppIcon
      v-if="todo.completed"
      icon="checked"
    />
    {{ todo.text }}
  </template>
</todo-list>
```

Use slots for:

- Content distribution (like layouts)
- Creating larger components by combining smaller components
- Default content in Multi-page Apps
- Providing a wrapper for other components
- Replace default component fragments

Use scoped slots for:

- Applying custom formatting/template to fragments of a component
- Creating wrapper components
- Exposing its own data and methods to child components

Pros

- Great for creating reusable and *composable* components
- Receiving properties from slot-scope is explicit

Cons

- Properties received through slot-scope can't be easily used in component script
 - However, you can pass those to methods inside the template as arguments



Questions?

Slots > Props

Composition > Configuration

With composition, you're less restricted by what you were building at first.

With configuration, you have to document everything and new requirements means new configuration.

TECHNIQUE

<Component :is="name">

PROBLEM

How to dynamically switch components
based on data?



```
<template>
  <div>
    <Component :is="clockType" v-bind="clockProps"/>
  </div>
</template>
```

<Component :is>

Becomes the component specified by the **:is** prop.

```
<script>
export default {
  components: { DigitalClock },
  computed: {
    clockType () {
      if (this.selectedClock === 'analog') {
        this.clockProps = {
          ...analogProps
        }
        return () => import(`./components/${this.compName}`)
      } else {
        return 'DigitalClock'
      }
    }
  }
}
// ...
}
```



```
<template>
  <div>
    <Component :is="clockType" v-bind="clockProps"/>
  </div>
</template>
```

```
<script>
export default {
  components: { DigitalClock },
  computed: {
    clockType () {
      if (this.selectedClock === 'analog') {
        this.clockProps = {
          ...analogProps
        }
        return () => import(`./components/${this.compName}`)
      } else {
        return 'DigitalClock'
      }
    }
  }
}
// ...
}
```

Pros

- Extremely powerful and flexible
- Easy to use
- Can accept props
- Can accept asynchronous components
- Can change into different components
- You can make a router-view out of it

Cons

- Got to handle props carefully

DESIGN PATTERN

Vendor Components Wrapper

```
<template>
```

```
<p>
```

```
<FontAwesome icon="water" />
```

```
<FontAwesome icon="earth" />
```

```
<FontAwesome icon="fire" />
```

```
<FontAwesome icon="air" />
```

```
</p>
```

```
</template>
```



```
<template>
```

```
<p>
```

```
<FontAwesome icon="water" />
```

```
<FontAwesome icon="earth" />
```

```
<FontAwesome icon="fire" />
```

```
<FontAwesome icon="air" />
```

```
</p>
```

```
</template>
```

```
<template>
```

```
<p>
```

```
<FontAwesome icon="water" />
```

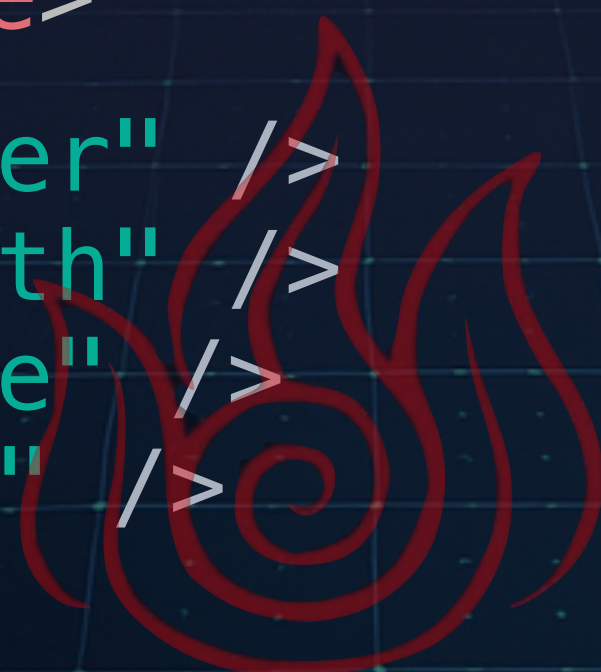
```
<FontAwesome icon="earth" />
```

```
<FontAwesome icon="fire" />
```

```
<FontAwesome icon="air" />
```

```
</p>
```

```
</template>
```



```
<template>
```

```
<p>
```

```
<FontAwesome icon="water" />
```

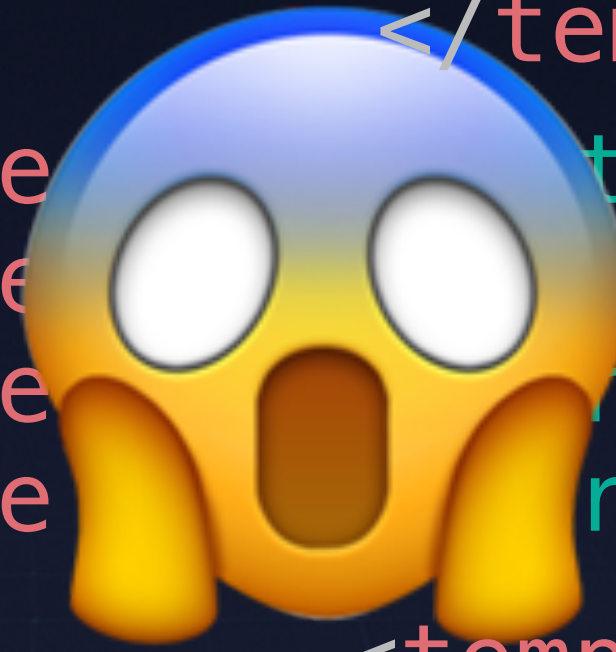
```
<FontAwesome icon="earth" />
```

```
<FontAwesome icon="fire" />
```

```
<FontAwesome icon="air" />
```

```
</p>
```

```
</template>
```



```
<template>
```

```
<p>
```

```
<FontAwesome icon="water" />
```

```
<FontAwesome icon="earth" />
```

```
<FontAwesome icon="fire" />
```

```
<FontAwesome icon="air" />
```

```
</p>
```

```
</template>
```



BaseIcon.vue

```
<template>
  <FontAwesomeIcon
    v-if="source === 'font-awesome'"
    :icon="name"
  />
  <span
    v-else
    :class="customIconClass"
  />
</template>
```

```
<template>
  <p>
    <BaseIcon icon="earth" />
    <BaseIcon icon="fire" />
    <BaseIcon icon="water" />
    <BaseIcon icon="air" />
  </p>
</template>
```

DESIGN PATTERN

Transparent Components

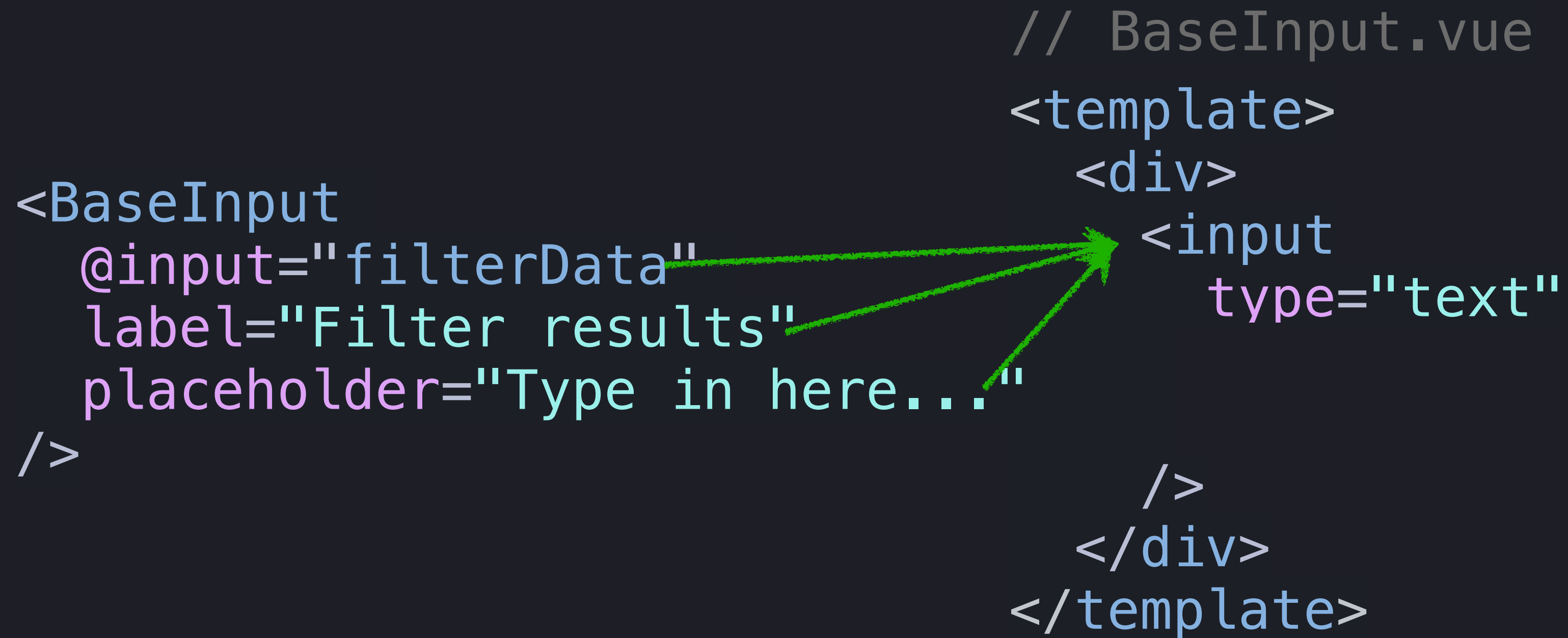
When passing props, listeners, and attributes...

```
// BaseInput.vue
<template>
  <div>
    <input
      type="text"
      v-bind="{ ...$attrs, ...$props }"
      v-on="$listeners"
    />
  </div>
</template>
```

When passing props, listeners, and attributes...

```
// BaseInput.vue
<template>
  <div>
    <input
      type="text"
    >
  </div>
</template>

<BaseInput
  @input="filterData"
  label="Filter results"
  placeholder="Type in here..."
/>
```

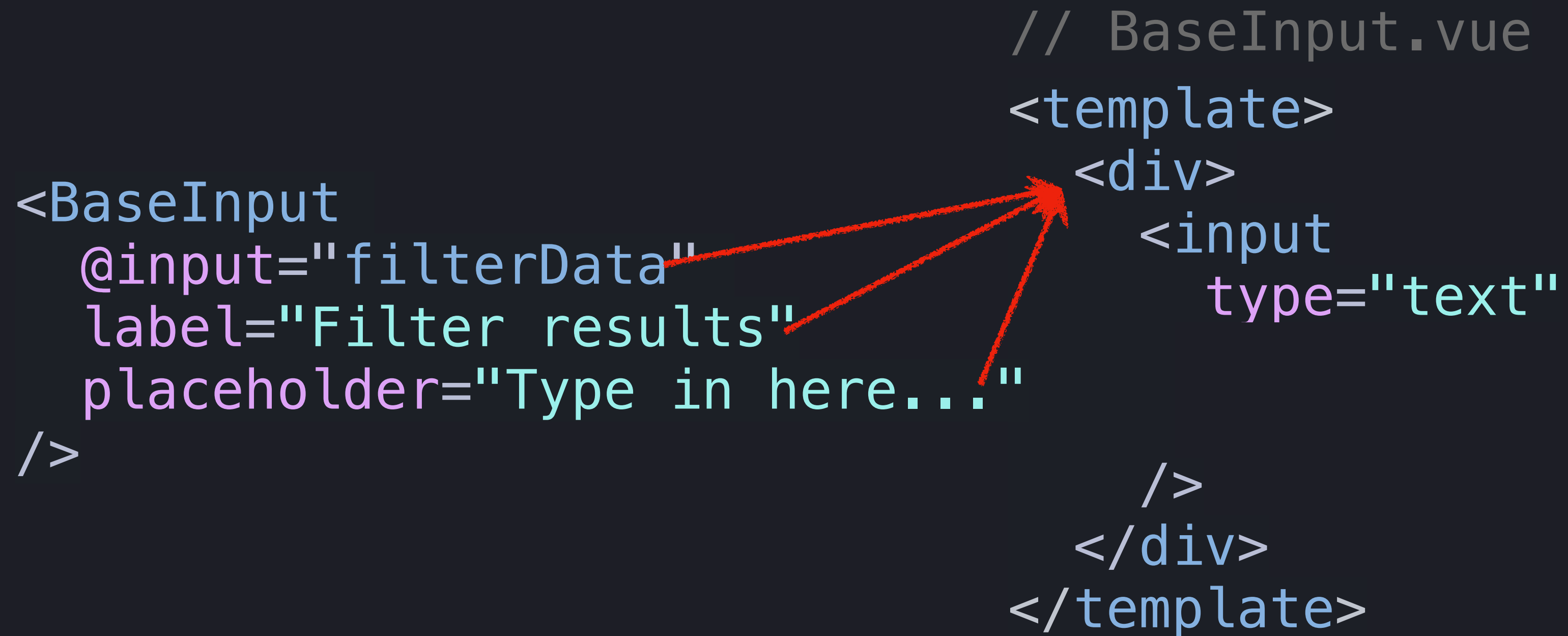


The diagram illustrates the flow of data from a parent component to a child component. On the left, the parent component's template contains a `<BaseInput>` tag with three attributes: `@input="filterData"`, `label="Filter results"`, and `placeholder="Type in here..."`. On the right, the child component's template (for `BaseInput.vue`) shows an `<input type="text">` tag. Three green arrows originate from the attribute values in the parent's tag and point to the `<input>` tag in the child's template, indicating that these values are passed as props to the child component.

When passing props, listeners, and attributes...

```
// BaseInput.vue
<template>
  <div>
    <input
      type="text"
    >
  </div>
</template>

<BaseInput
  @input="filterData"
  label="Filter results"
  placeholder="Type in here..."
/>
```



The diagram illustrates the flow of data from a parent component to a child component. On the left, the parent component's closing tag `</BaseInput>` is shown with three attributes: `@input="filterData"`, `label="Filter results"`, and `placeholder="Type in here..."`. On the right, the child component's template is shown, starting with `<div>` and containing an `<input type="text">` element. Three red arrows originate from the attribute values in the parent tag and point to the `<div>` tag in the child template, indicating that these attributes are passed to the child component.

When passing props, listeners, and attributes...

```
<template>
  <div>
    <input
      type="text"
      v-bind="{ ...$attrs, ...$props }"
      v-on="$listeners"
    />
  </div>
</template>
<script>
export default {
  inheritAttrs: false,
  // ...
}
</script>
```

Both props and attributes, as well as all listeners will be passed to this element instead.

Prevent Vue from assigning attributes to top-level element

When passing props, listeners, and attributes...

```
<template>
  <div>
    <input
      type="text"
      v-bind="{ ...$attrs, ...$props }"
      v-on="$listeners"
    />
  </div>
</template>

<script>
export default {
  inheritAttrs: false,
  // ...
}
</script>
```

When passing props, listeners, and attributes in Vue 3...

```
<template>
  <div>
    <input
      type="text"
      v-bind="{ ...$attrs, ...$props }"
      v-on="$listeners"
    />
  </div>
</template>

<script>
export default {
  inheritAttrs: false,
  // ...
}
</script>
```

When passing props, listeners, and attributes in Vue 3...

```
<template>
  <div>
    <input type="text" v-bind="$attrs" />
  </div>
</template>
```

```
<script>
export default {
  inheritAttrs: false,
  // ...
}
</script>
```



Questions?



CODE / EXPERIMENT



In the boilerplate

- Create a BaseInputText component using the transparent wrapper pattern
- Use dynamic components to toggle between different components

In your app

- Look for opportunities to rename your components
- See if you can optimize any props, emits or slots usage
- Refactor a component using the vendor wrapper pattern

DISCUSSION

When to Refactor Your Components

*Premature optimization is the
root of all evil (or at least most of
it) in programming.*

– Donald Knuth

PRINCIPLE

Data Driven Refactoring

PRINCIPLE

Data Driven Refactoring

Signs you need more components

- When your components are hard to understand
- You feel a fragment of a component could use its own state
- Hard to describe what what the component is actually responsible for

PRINCIPLE

Data Driven Refactoring

How to find reusable components?

- Look for v-for loops
- Look for large components
- Look for similar visual designs
- Look for repeating design interface fragments
- Look for multiple/mixed responsibilities
- Look for complicated data paths



Questions?



LUNCH BREAK

Be back at 12:50PM!